

期末考试

三类题型：

- 选择：15 * 2 （有2、3道nosql）
- 问答：8题40分（理解的内容，有2道nosql如下）
 - 为什么需要nosql，和sql之间的关系
 - 四个nosql的基本构造、原理、上下文
- 综合：3 * 10
 - ER图
 - sql（给条件，写sql语句和关系代数表达式）
 - 规范化（给依赖条件，是什么范式，化简为更高级范式）

未提交的作业：

- 出分之前邮件发送视为补交，简言之就是没交的尽快交
- 但补交分数会有一定折扣

往年题目：

[2020-数据管理基础-exam-考试题目考点整理 - SpriCoder的博客](#)

知识点整理

人工管理、文件系统与数据库管理数据

	管理者	面向对象	数据共享	数据结构化	数据独立	数据控制能力
人工管理	用户	某一应用	无共享，冗余大	无结构	不独立，依赖于程序	应用程序自己控制
文件系统	文件系统	某一应用	共享差，冗余大	记录内有结构，整体无结构	独立性差	应用程序自己控制
数据库管理	DBMS	现实世界	共享性好	整体结构化	独立性高	DBMS

- **数据结构化**，主要特征之一，本质区别。整体结构化（面向对象、数据之间有联系）
- 数据**共享性好**有利于减少冗余，避免数据间不相容和不一致，易于扩充系统
- 数据独立是指数据的使用（应用程序）与数据的说明（数据的组织结构与存储方式）分离
- 数据库的数据**独立性高**：物理独立性高（物理存储独立）+ 逻辑独立性高（逻辑结构独立），有DBMS二级映象保证
- 数据**由DBMS统一管理控制**：数据安全性保护、完整性检查、并发控制、数据库错误恢复

几个概念

- **数据**：数据库中存储的基本对象，是描述事物的符号记录
- **数据库DB**：是**长期储存在计算机内、有组织、可共享**的大量数据的**集合**
 - 基本特征见上表6
- **数据库管理系统DBMS**：位于用户和操作系统之间的一层数据管理软件，基础软件，是大型复杂的软件系统
 - 科学地组织和存储数据、高效地获取和维护数据
 - 主要功能有：
 1. 数据定义功能（数据定义语言DDL）
 2. 数据组织、存储和管理

3. 数据操纵功能（数据操纵语言DML）、增删改查

4. 数据库的事务管理和运行管理

数据库建立、运行、维护是统一管理控制

数据安全性、完整性、多用户对数据的并发使用

发生故障后的系统恢复

5. 数据库的建立和维护功能（数据装载、转换；转储恢复；重组织；性能监视、分析）

6. 其他（和其他软件系统间通信、DBMS之间的数据转换、异构数据库间的互访和互操作）

- **数据库系统DBS**：数据库 + 数据库管理系统（及其应用开发工具） + 应用程序 + 数据库管理员

数据模型

- 数据模型是对现实世界数据特征的抽象，用以抽象画、表示和处理现实世界中的数据和信息（要比较真实地模拟、容易理解、便于在计算机上实现），**数据模型是数据库系统的核心和基础**
- 模型：
 1. 概念模型（用户观点建模）
 2. 逻辑模型（计算机系统观点建模）

物理模型（对数据的最底层抽象，在系统内部的表示和存取方法）
- 两步抽象
 - 第一步：现实世界客观对象——概念模型（现实世界抽象为信息世界）
 - 第二步：概念模型——特定的DBMS支持的数据模型（信息世界转换为机器世界）
- 数据模型的组成要素：数据结构+数据操作+数据完整性约束条件

概念模型的几个名词解释

- 实体：**客观存在并可互相区别**的事物称为实体
- 属性：实体所具有的某一特性称为属性，一个实体可以由若干属性来刻画
- 码：**唯一标识实体的属性集称为码**
- 实体型：用实体名及其属性名集合来抽象和刻画同类实体称为实体型
- 实体集：同一类型实体的集合

- 联系：现实世界中事物内部以及事物之间的联系在信息世界中反映为实体（型）内部的联系和实体（型）间的联系
-

数据库系统的结构

- 模式与实例：
 - 模式是数据库逻辑结构与特征的描述，是型的描述，反映的是数据的结构及其联系，是相对稳定的
 - 实例则反映数据库某一时刻的状态，是模式的一个具体值，实例随数据库中数据的更新而变动
- 数据库系统的三层模式结构
 -
 - 模式（逻辑模式）是所有用户的公共数据视图，即**全局逻辑结构是数据库的中心与关键**。一个数据库只有一个模式。模式是数据库系统模式结构的中间层，与数据的物理存储细节和硬件环境无关，与具体的应用程序、开发工具即高级程序设计语言无关。**最先需要确定**
 - 外模式（子模式、用户模式）是数据库用户使用的局部数据的逻辑结构和特征描述。是数据库用户的数据视图，是与应用有关的。它介于模式与应用之间，一个模式对应多个外模式（通常是子集关系），一个外模式也对应多个应用。外模式的存在保证数据库安全性，即用户只能看见和访问所对应的外模式中的数据
 - 内模式（存储模式）是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式，一个数据库只有一个内模式。独立于具体存储设备。

三级模式与二级映象

- 上述三级模式是对数据库的三个抽象级别
- 二级映象
 - 外模式 / 模式映象
 - 映象定义通常包含在各自外模式的描述中
 - 保证数据的逻辑独立性：模式改变，对映象做变更，使得外模式不变，应用程序根据外模式编写，也不变
 - 模式 / 内模式映象
 - 该映象定义通常包含在模式描述中
 - 保证数据的物理独立性：同上，只要更改映象，不用改上层模式

数据库系统的组成

- 硬件平台及数据库 + 软件 + 人员
 - 人员：数据库管理员DBA + 系统分析员和数据库设计人员 + 应用程序员 + 最终用户

关系-关系模型-关系数据库

- **域**：是一组具有相同数据类型的值的集合，属性的取值范围来自某个域（例如整数、实数.....）
- **笛卡尔积**：给定一组域，允许其中某些域相同，则是所有域的所有取值的一个组合

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) | d_i \in D_i, i = 1, 2, \dots, n\}$$

- **元组**：表中的一行即为一个元组，即笛卡尔积中的内一个元素 $(d_1, d_2, \dots, d_n)$
- **分量**：元组中的一个属性值，即笛卡尔积元素中的每一个值 d_i
- **基数**：笛卡尔积元素的个数
- 笛卡尔积可以表示为一张表：每行对应一个元组，每列对应一个域

- **关系**：一个关系对于通常说的一张表，在笛卡尔积的定义下，是一个**子集**
- **属性**：表中的一列即为一个属性，给每一个属性起一个名称即属性名（属性是为了对相同域加以区分），n目关系有n个属性
- **码**：
 - **候选码**：若关系中的某一属性组的值能唯一地标识一个元组，而其子集不能，则称该属性组为候选码
 - **主码**：若一个关系有多个候选码，则选定其中一个为主码
 - **主属性**：候选码的诸属性称为主属性，不包含在任何侯选码中的属性称为非主属性
 - **外码**：设F是基本关系R的一个或一组属性，但不是关系R的码。如果F与基本关系S的主码Ks相对应，则称F是R的外码；R称为参照关系，S称为被参照关系
- **关系模式**：对关系的描述，关系名（属性1，属性2，属性3），关系模式是型，关系是值
 - $R(U, D, DOM, F)$ ，简记为 $R(U)$
 - R：关系名
 - U：组成该关系的属性名集合
 - D：U中属性来自的域

- DOM: 属性向域的映象集合
- F: 属性间数据的依赖关系的集合
- 关系模型的优缺点:
 - 优点: 建立在严格的数学模型上, 概念单一, 关系模型的存取路径相对用户透明 (更高数据独立性和安全保密性; 简化程序员工作和数据库开发建立工作)
 - 缺点: 存取路径相对用户透明 (查询效率不如格式化数据模型), 为提高性能需要对查询请求优化, 增加DBMS开发难度
- 关系的完整性约束: 实体完整性、参照完整性、用户定义的完整性
 - 实体完整性: 主属性**不能取空值**, 对应于现实世界中实体是可区分的
 - 参照完整性: 外码必须**取空值, 或者等于被参照关系中某个元组的主码值**
 - 用户定义的完整性: 针对于某一具体关系数据库的约束条件, 反映具体应用所涉及数据必须满足的语义要求
- 关系操作的特点: 都是集合式操作, 对象和结果都是集合, 一次一集合
 - 查询: **选择、投影、连接、除、并、差、交、笛卡尔积** (5种基本操作)
 - 更新: 插入、删除、修改
- 关系代数: 一种抽象的查询语言, 用对关系的运算来表达查询
 - 集合运算符: 行的角度
 - 专门的关系运算符: 不仅涉及行, 还涉及列
- 记号

设关系模式为 $R(A_1, A_2, \dots, A_n)$

- 它的一个关系设为 R
- $t \in R$ 表示 t 是 R 的一个元组
- $t[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量
- 若 $A = \{A_1, A_2, \dots, A_k\}$, 其中 $A_1, A_2, \dots, A_k$ 是 $A_1, A_2, \dots, A_n$ 中的一部分, 则 A 称为属性列或属性组。
- $t[A] = \{t[A_1], t[A_2], \dots, t[A_k]\}$ 表示元组 t 在属性列 A 上诸分量的集合。
- $\bar{A}$ 则表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_1, A_2, \dots, A_k\}$ 后剩余的属性组。

R 为 n 目关系, S 为 m 目关系。

- $t_r \in R, t_s \in S, t_r t_s$ 称为元组的连接。
- $t_r t_s$ 是一个 $n + m$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组。

给定一个关系 $R(X, Z)$, X 和 Z 为属性组。

- 当 $t[X] = x$ 时, x 在 R 中的象集 (Images Set) 为:

$$Z_x = \{t[Z] | t \in R, t[X] = x\}$$

- 它表示 R 中属性组 X 上值为 x 的诸元组在 Z 上分量的集合

- 并: $R \cup S$, 属于 R 或属于 S 的元组组成
- 差: $R - S$, 属于 R 而不属于 S 的所有元组组成
- 交: $R \cap S$, 既属于 R 又属于 S 的元组组成
- 广义笛卡尔积: 其实就是排列组合, 变成 $k_1 * k_2$ 个 $(m + n)$ 列元组的集合
- 选择: $\sigma_F(R)$, 在关系 R 中选择满足给定条件 F 的诸元组
- 投影: $\pi_A(R)$, 从 R 中选择出若干属性列 (A 为 R 中若干属性列) 组成新的关系

- 连接：， 从两个关系的笛卡尔积中选取属性间满足一定条件（ $A\theta B$ ）的元组
 - 等值连接： θ 为 “=”， 选取两属性值相等的那些元组
 - 自然连接：特殊的等值连接， **比较的分量是相同属性组**， 并在结果中把**重复的属性列去掉**
 - 外连接：自然连接保留悬浮元组， 左外连接右外连接（保留左边或右边关系中的悬浮元组）
 - 悬浮元组就是两个关系的相同属性组， 但是只有一个关系里的分量值和另一个关系里的值都不等（即没有比较出不同的机会）， 就将整个元组保留
- 除： $R \div S$ ， R 中的元组在非相同属性组上分量值的象集包含 S 在相同属性组上投影的集合， 贴张图理解一下

R

A	B	C
a1	b1	c2
a2	b3	c7
a3	b4	c6
a1	b2	c3
a4	b6	c6
a2	b2	c3
a1	b2	c1

S

B	C	D
b1	c2	d1
b2	c1	d1
b2	c3	d2

R ÷ S

A
a1

- 在关系R中，A可以取四个值{a1, a2, a3, a4}
 - a1的象集为 {(b1, c2), (b2, c3), (b2, c1)}
 - a2的象集为 {(b3, c7), (b2, c3)}
 - a3的象集为 {(b4, c6)}
 - a4的象集为 {(b6, c6)}
- S在(B, C)上的投影为

{(b1, c2), (b2, c1), (b2, c3)}
- 只有a1的象集包含了S在(B, C)属性组上的投影

所以 R÷S={a1}

SQL

- SQL是结构化查询语言， 是关系数据库的标准语言， 是一个通用的、 功能性极强的关系数据库语言
- SQL与关系数据库的三级模式
 - 基本表： 本身独立存在的表， 一个关系对应一个基本表， 若干基本表对应一个存储文件， 一个表带若干索引
 - 存储文件
 - 视图： 从一个或几个基本表中导出的表， 数据库只存放视图的定义， 而不存放其数据， 是一个虚表， 视图纸上可再定义视图

1. 数据定义

1.1 模式

- 定义模式：其实是定义了一个命名空间，在此空间中可以定义该模式包含的数据库对象（基本表、视图、索引）

```
1 CREATE SCHEMA <模式名>
2 AUTHORIZATION <用户名> [ <表定义子句> | <视图定义子句> | <授权定义子句> ]
```

- 没有指定模式名时，隐含为用户名
- 删除模式：两种删除方法——级联、限制

```
1 DROP SCHEMA <模式名> <CASCADE | RESTRICT>
```

- 级联：把该模式中所有数据库对象全部删除
- 限制：如果该模式下有下属数据库对象，则拒绝删除语句的执行

1.2 基本表

- 定义基本表

```
1 CREATE TABLE <表名>
2     (<列名> <数据类型> [ <列级完整性约束条件> ]
3     [, <列名> <数据类型> [ <列级完整性约束条件> ] ]
4     .....
5     [, <表级完整性约束条件> ] );
```

- 完整性约束条件涉及多个属性列时，需要定义在表级上
- PPT中涉及到的一些完整性约束条件
 - 定义在列级的：PRIMARY KEY、UNIQUE、NOT NULL
 - 定义在表级的：PRIMARY KEY(Sno, Cno)、FOREIGN KEY (Sno) REFERENCE Student(Sno)
- 修改基本表

```
1 ALTER TABLE <表名>
2 [ ADD [ COLUMN ] <新列名> <数据类型> [ 完整性约束 ] ]
3 [ ADD <表级完整性约束> ]
4 [ DROP [ COLUMN ] <列名> [ CASCADE | RESTRICT ] ]
5 [ DROP CONSTRAINT <完整性约束名> [ RESTRICT | CASCADE ] ]
6 [ ALTER COLUMN <列名> <数据类型> ] ;
```

- DROP时由级联和限制，级联自动删除引用改了的的其他对象，限制有引用就拒绝
- DROP CONSTRAINT删除指定的完整性约束条件

- ALERT COLUMN修改列的定义
- 删除基本表

```
1 | DROP TABLE <表名> <CASCADE | RESTRICT>
```

1.3 索引

- 索引的建立是为了加快查询速度。是由数据库管理员或表拥有着建立的，由关系数据库管理系统自动维护的。用户不必也不能显式选择索引
- 常见索引：顺序文件上的索引、B+树索引、散列索引、位图索引
- 建立索引

```
1 | CREATE [UNIQUE] [CLUSTER] INDEX <索引名>
2 | ON <表名>(<列名>[<次序>][,<列名>[<次序>] ]...);
```

- UNIQUE：每一个索引值对应唯一数据记录
- CLUSTER：是否为聚簇索引
- 次序：升序ASC、降序DESC，缺省为升序
- 修改索引

```
1 | ALTER INDEX <旧索引名> RENAME TO <新索引名>
```

- 删除索引

```
1 | DROP INDEX <索引名>
```

数据字典：是关系数据库管理系统内部的一组系统表，记录当前数据库所有定义信息，包括

- 关系模式定义、视图定义、索引定义、完整性约束定义、各类用户操作权限、统计信息等
- 执行SQL数据定义语句实际就是在更新数据字典表中的相应信息

2. 数据查询

2.1 单表查询

```
1 | SELECT [ALL|DISTINCT] <目标列表表达式>[,<目标列表表达式>]...
2 | FROM <表名或视图名>[,<表名或视图名> ]... | (SELECT 语句) [AS] <别名>
3 | [WHERE <条件表达式>]
4 | [GROUP BY <列名1> [HAVING<条件表达式>]]
5 | [ORDER BY <列名2> [ASC|DESC]];
```

- ALL 与 DISTINCT关键词：输出所有还是去掉重复行，缺省为ALL

- 根据**WHERE子句的条件表达式**从**FROM子句指定**的基本表、视图或派生表中找出满足条件的元组，再按SELECT语句中目标列表表达式**选出元组中的属性值**形成出结果
 - 目标列表表达式也可以是表达式，即经过计算或操作的值，也可以设置列别名 *Sname NAME*
- 如果有**GROUP BY语句**，则将结果按照<列名1>的值进行分组，属性值相等的元素为一个组。通常会在每组中作用聚集函数。
 - 如果GROUP语句带**HAVING语句**，则只有满足指定条件的组才予以输出
- 如果有**ORDER BY语句**，则按照<列名2>的值进行升序/降序排列
- 查询表达式

- [NOT] LIKE '<匹配串>' [ESCAPE '<换码字符>']
 - 匹配串可以含有通配符 % (任意长度字符串) 和 _ (任意单个字符)
 - ESCAPE '\ ' 表示 \ 为换码字符，用于匹配 %、_ 这些字符

查询以"DB_"开头，且倒数第3个字符为 i 的课程的具体情况。

```
WHERE Cname LIKE 'DB\_ %i\_ ' ESCAPE '\ ';
```

- 设计到空值的查询需要注意使用**IS NULL 或 IS NOT NULL**，IS不能用=替代
- 多重条件查询：AND优先级高于OR，用括号改变优先级
- GROUP BY语句与聚集函数的使用
 - 聚集函数：COUNT()、SUM()、AVG()、MAX()、MIN()
 - 关键字ALL、DISTINCT
 - 如果未对查询结果分组（即没有GROUP BY语句），聚集函数将作用于整个查询结果
 - 对查询结果分组后，聚集函数将分别作用于每个组
 - HAVING相当于分组之后再进行进一步的筛选，不能用WHERE
 - HAVING短语与WHERE子句的区别：
 1. 作用对象不同
 2. WHERE子句作用于基表或视图，从中选择满足条件的元组
 3. HAVING短语作用于组，从中选择满足条件的组。

2.2 连接查询

- 设计到两表以上的查询

- 1 | [**<表名1>.**] **<列名1>** **<比较运算符>** [**<表名2>.**] **<列名2>**
- 2 | [**<表名1>.**] **<列名1>** **BETWEEN** [**<表名2>.**] **<列名2>** **AND** [**<表名2>.**] **<列名3>**

- 执行过程

- 嵌套循环法：就是两层循环
- 排序合并法：先排序再循环，中断循环条件
- 索引连接：对表2连接字段建立了索引
- sql可以同时执行选择和连接操作 WHERE子句是有连接谓词和选择谓词组成的符合条件

- 自身连接：需要给表取个别名，例子

```
1 | SELECT FIRST.Cno, SECOND.Cpno
2 |     FROM Course FIRST, Course SECOND
3 |     WHERE FIRST.Cpno = SECOND.Cno;
```

- 外连接（左外、右外）

```
1 | SELECT Student.Sno, Sname, Ssex, Sage, Sdept, Cno, Grade
2 |     FROM Student LEFT OUT JOIN SC ON (Student.Sno=SC.Sno);
```

- 多表连接：就是FROM的表多一点，WHERE的连接谓语多一点

2.3 嵌套查询

- 允许多层嵌套；子查询不能使用ORDER BY子句；嵌套查询部分可以使用连接运算替代（谨慎使用嵌套查询）
- 求解方法：
 - 不相关子查询：由里向外，逐层处理
 - 相关子查询（子查询的查询条件依赖于父查询）：去外层查询中表的每一个元组，根据子查询WHERE子句是否为真决定是否将其放入结果表
- 带有ANY 和 ALL的子查询，可以使用聚集函数来实现
- 带有EXISTS的子查询
 - 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。
 - 若内层查询结果非空，则外层的WHERE子句返回真值
 - 若内层查询结果为空，则外层的WHERE子句返回假值
 - 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换
 - 使用存在量词替换全称量词：

- 使用存在谓词实现逻辑蕴涵，例子：

2.4 集合查询

- UNION：将多个查询结果合并，去掉重复元组
- UNION ALL：将多个查询结果合并，保留重复元组
- INTERSECT：取交集
- EXCEPT：取差集

2.5 派生表查询

- 就是子查询在FROM语句中

3. 数据更新

3.1 插入

```
1 INSERT
2 INTO <表名> [(<属性列1>[,<属性列2 >...])
3 VALUES (<常量1> [,<常量2>]... );
```

- 属性列的顺序可与表定义中的顺序不一致
- 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
- 指定部分属性列：插入的元组在其余属性列上取空值

```
1 INSERT
2 INTO <表名> [(<属性列1> [,<属性列2>...  )]
3 子查询;
```

- 插入子查询结果

3.2 修改

```
1 UPDATE <表名>
2 SET <列名>=<表达式>[,<列名>=<表达式>]...
3 [WHERE <条件>];
```

- 关系数据库管理系统在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则

3.3 删除

```
1 DELETE
2 FROM <表名>
3 [WHERE <条件>];
```

- 有WHERE，删除满足WHERE的元组
- 没有WHERE，删除全部元组，但是表仍然存在字典中

4. 空值

- 空值的理解：
 1. 这个属性应该有一个值，但是目前不知道具体的值
 2. 这个属性不应该有值
 3. 处于某些原因不便填写
- 产生：
 1. 插入数据时，没有指定的值默认为空值
 2. 显式地修改某个值为空值
- 判断：IS NULL 或 IS NOT NULL
- 约束：
 1. 有NOT NULL约束条件的不能取空值
 2. 加了UNIQUE限制的属性不能取空值
 3. 码属性不能取空值
- 运算：
 1. 空值与另外一个值的算数运算为空值
 2. 比较运算为UNKNOWN（有了UNKNOWN，二值逻辑就变为三值逻辑）
 - AND仍然有T为T，OR仍然为由T为T（短路）
 - 但是需要两个值同时计算的话，有U为U

5. 视图

- 视图是一个虚表。是从几个基本表（或视图）中导出的表，只存放定义，不存放数据

5.1 建立视图

```
1 CREATE VIEW <视图名> [(<列名> [, <列名>] ...)]
2 AS <子查询>
3 [WITH CHECK OPTION];
```

- WITH CHECK OPTION：对视图进行UPDATE，INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件（即子查询中的条件表达式）

- 属性列名全部省略（由子查询中字段组成）或全部给出，以下需要全部给出
 - 某个目标列是聚集函数或列表表达式
 - 多表连接时选出了几个同名列作为视图的字段
 - 需要在视图中为某个列启用新的更合适的名字
- 行列子集视图：视图从单个表中导出，并且只是去掉了基本表的某些行和列，但保留了主码

5.2 删除视图

```
1 DROP VIEW <视图名>[CASCADE];
```

- 从字典中删除指定视图定义，级联一并删除由其导出的视图
- 删除基表时则需要显式删除其导出的所有视图定义

5.3 查询视图

- 用户视角与查询基本表相同
- 数据库管理系统实现：有效性检查 + 转换为等价的对基本表的查询 + 执行修正后的查询

5.4 更新视图

- 删除和插入操作都是转换为对对应的基本表进行操作

```
1 INSERT
2 INTO <视图名> [( <属性列1> [, <属性列2 > ... ])
3 VALUES ( <常量1> [, <常量2> ] ... );
4
5 DELETE
6 FROM <视图名>
7 WHERE <条件表达式>
```

- 不同系统对视图的更新有不同限制，在于其是否能够有意义地转换为对相应基本表的更新

5.5 视图的作用

- 简化用户操作：基于多表连接、复杂嵌套、导出属性的视图
- 使得用户能以多种角度看待同一数据：以适应数据库共享的需要
- 对重构数据库提供了一定程度的逻辑独立性
- 对机密数据提供安全保护：对不同用户定义不同视图
- 可以更清晰的表达查询

恭喜你过完了sql!

数据库安全性

- 数据的共享必然会带来数据库的安全性问题，所以共享不能是无条件的共享
- **安全性是指保护数据库以防止不合法的使用造成的数据泄露、更改或破坏；系统安全保护措施有效性是性能指标之一**
- 不安全要素：
 1. 非授权用户对数据库的恶意存取和破坏
 2. 重要敏感数据泄露
 3. 安全环境的脆弱性（与计算机系统的安全性相关）
- TCSEC/TDI安全等级划分（可信计算机系统评估准则）
 - D：兜底的，就不符合更高级的都是D
 - C1：非常初级，用户数据分离，实现自主存取控制（DAC）
 - C2：安全产品最低档次，受控存取保护，注册审计
 - B1：标记安全保护，对数据加标记，对标记的主客体实施强制存取控制MAC、审计等
 - B2：结构化保护，形式化的安全策略模型对所有主客体实施DAC和MAC
 - B3：安全域，TCB必须满足监控器要求，审计跟踪能力强，系统恢复过程
 - A1：验证设计，B3的同时给出系统形式化设计说明和验证来确保安全保护真正实现
- CC（Common Criteria 通用标准）
 - 提出国际公认的表述信息技术安全性的结构
 - 把信息产品的安全要求分为：安全功能要求 + 安全保证要求
 - 组成：在上面两个要求上还加了“简介和一般模型”
 - 根据安全保证要求，划分了7个等级，如下
- 计算机安全模型
 - 安全措施一级一级层层设置
 - DBMS安全控制模型（关注这里的多层访问控制）
- 存取控制子系统：定义用户权限 + 合法权限检查，DBMS与数据字典的等级与检查

DAC（自主存取控制）：C2级

- 权限：
 - 用户对不同对象有不同的存取权限
 - 不同用户对同一对象也有不同权限
 - 用户还可以将其拥有的权限转授给其他用户
 -
- 自主存取控制的缺点：可能存在数据的“无意泄漏”
 - 原因：这种机制仅仅通过对数据的存取权限来进行安全控制，而数据本身并无安全性标记
 - 解决：对操作系统控制下的所有主客体实施强制存取控制策略（MAC，见下）
- 授权实现：

- 用户权限由**数据库对象**和**操作类型**组成
- GRANT和REVOKE语句实现操作权限的手语与回收
 - 数据库管理员有所有权限和授予权限的能力
 - 用户有自己建立对象的所有权限，并可以授予给其他用户
 - 被授权用户如果有“继续授权”许可，也可授权给其他用户

- ```
1 GRANT <权限>[,<权限>]...
2 ON <对象类型> <对象名>[,<对象类型> <对象名>]...
3 TO <用户>[,<用户>]...
4 [WITH GRANT OPTION];
```

- 不允许循环授权
- 对属性列的授权必须明确给出属性列名，如UPDATE(Sno)

- ```
1 REVOKE <权限>[,<权限>]...
2 ON <对象类型> <对象名>[,<对象类型><对象名>]...
3 FROM <用户>[,<用户>]...[CASCADE | RESTRICT];
```

- 级联回收，同时回收直接或间接从其获得的权限

- GRANT和REVOKE语句面向用户对数据的权限；而对数据库模式的权限（区别见上权限表）则由数据库管理员在创建用户时实现

- ```
1 CREATE USER <username>
2 [WITH] [DBA | RESOURCE | CONNECT];
```

- 只有超级用户才能创建新用户
- 三种权限

- 默认为CONNECT



- 数据库角色：是被命名的一组域数据库操作相关的权限，可以为一组具有相同权限的用户创建一个角色来简化授权过程。

```

1 CREATE ROLE <角色名>
2
3 GRANT <权限>[,<权限>]...
4 ON <对象类型>对象名
5 TO <角色>[,<角色>]...
6
7 GRANT <角色1>[,<角色2>]...
8 TO <角色3>[,<用户1>]...
9 [WITH ADMIN OPTION]

```

- 指定了WITH ADMIN OPTION则获得某种权限的角色或用户还可以把这种权限授予其他角色
- 一个角色的权限：直接授予这个角色的全部权限加上其他角色授予这个角色的全部权限

## MAC (强制存取控制, B1级)

- 实现：
  - 每个数据被表以一定密级
  - 每个用户被授予某一个级别的许可证
  - 对于任意对象，只要具有合法许可证的用户才可以存取
  - 用户不能直接感知或进行控制
- 主客体
  - 主体是系统中的活动实体，实际用户、代表用户各进程
  - 客体是系统中的被动实体，受主体操纵
- 敏感度标记 Label
  - 级别：绝密TS、机密S、可信C、公开P
  - 主体敏感度标记称为许可证级别
  - 客体敏感度标记称为密级
- 规则：
  - 仅当主体的许可证级别大于或等于客体的密级时，该主体才能读取相应的客体
  - 仅当主体的许可证级别小于或等于客体的密级时，该主体才能写相应的客体

为什么？是为了放置密级从高到低流

书上的例子，如果TS密级的主体恶意将一个TS密级的数据降级为P，然后将其写回，则造成TS密级数据的泄漏，所以要放置主体对低于自己密级的数据的写权限

换个理解方式就是最低等级的士兵可以决定自己写入的数据给谁看，比如给排长，给司令，但是司令不能擅自决定机密文件给谁看，所以是对高许可证的主体进行的约束

应试技巧：第一条符合直觉、写反着来

- 实现强制存取控制时要首先实现自主存取控制（DAC + MAC）
  - 原因：较高安全性级别提供的安全保护要包含较低级别的所有保护
  - 先进行DAC，在进行MAC

## 其他机制

- 视图机制：为不同用户定义不同视图，把要保密的数据对无权存取的用户隐藏起来，间接实现支持存取谓词的用户权限定义
- 审计：启用一个专用的审计日志将用户对数据库的操作记录，审计员利用日志监控行为；C2级别以上必备
  - 可选性：很费时间空间、可以灵活打开关闭、用于安全性需求较高部门
  - 审计事件：服务器时间、系统权限、语句时间、模式对象时间
  - 审计功能：
    - 审计级别：用户级、系统级
    - 审计语句：AUDIT、NOAUDIT，设置和关闭审计功能

```
1 | AUDIT <操作名>
2 | ON <对象名>
```

- 数据加密：根据算法将原始数据转为不可直接识别的格式（明文——密文）
  - 存储加密：透明、非透明
  - 传输加密：链路、端到端
- 其他：推理控制、隐蔽信道、数据隐私保护

## 数据库完整性

- 数据库的完整性是指数据的正确性（符合现实）和相容性（同一对象在不同关系表中的数据是符合逻辑的）
- 为了维护数据库的完整性，DBMS必须实现以下功能：
  1. 提供定义完整性约束条件的机制

- SQL标准使用了一系列概念来描述完整性，包括关系模型的实体完整性、参照完整性和用户定义完整性
- 2. 提供完整性检查的方法
- 3. 进行违约处理

## 实体完整性

- 主属性**不能取空值**，CREATE TABLE 中用 PRIMARY KEY 定义
- 单属性定义在列级表级都行，多属性只能表级
- 完整性检查与违约处理
  - 插入或对主码进行检查
    - 1. 检查主码值是或否唯一（全表扫描或建立索引），不唯一拒绝插入
    - 2. 检查主码属性是否为空，有为空就拒绝

## 参照完整性

- 外码必须**取空值，或者等于被参照关系中某个元组的主码值**
- 在CREATE TABLE中用FOREIGN KEY短语定义哪些列为外码，用REFERENCES短语指明这些外码参照哪些表的主码
- 参照完整性将两张表联系起来，两张表的操作都可能破坏参照完整性，检查：
  - 级联操作：删除或修改参照表中的所有造成不一致的元组
  - 设为控制，将参照表中不一致的元组对于属性设为空
  - 在定义时，增加语句 ON <操作> [CASCADE|NO ACTION]

## 用户定义的完整性

- 针对某一具体应用的数据必须满足的语义要求，包括属性上的和元组上的约束条件
- 对于用户定义完整性的检查有DBMS完成，不需要程序承担
- 创建与修改
  - <完整性约束条件>包括NOT NULL、UNIQUE、PRIMARY KEY短语、FOREIGN KEY短语、CHECK短语等
  - 可以直接CREAT TABLE式在属性上或元组上使用完整性约束条件定义，或使用CONSTRAINT子句

1 | **CONSTRAINT** <完整性约束条件名><完整性约束条件>

- 修改是删除原来的约束条件，增加新的约束条件

## 断言

- 通过CREAT ASSERTION语句，通过声明性断言来指定更具一般性的约束
- 可以使用断言定义设计多个表或聚集操作的比较复杂的完整性约束，断言创建后，对其设计操作都还会触发检查，断言不真的操作被拒绝

- ```
1 CREAT ASSERTION<断言名><CHECK子句>
2 DROP ASSERTION<断言名>
```

触发器（事件-条件-动作规则）

- Trigger是用户定义在**关系表**（不能是视图）上的一类由事件驱动的特殊过程（保存在数据库服务器中、任何对表的增删改操作都会自动触发相应触发器）

- ```
1 CREATE TRIGGER <触发器名>
2 {BEFORE | AFTER} <触发事件> ON <表名>
3 REFERENCING NEW|OLD ROW AS<变量>
4 FOR EACH {ROW | STATEMENT}
5 [WHEN <触发条件>]<触发动作体>
```

- 只有表的拥有者才可以在表上创建触发器
- 同一模式下，触发器名唯一，触发器名与表名必须在统一模式下
- <触发器事件>：可以是INSERT、DELETE、UPDATE的组合，也可以是UPDATE OF<触发列、...>即进一步指明修改哪些列时激活触发器
- {AFTER|BEFORE}是触发时机
- {ROW|STATEMENT}是触发器类型，行级、语句级
- 触发条件，在被激活后满足一定条件才执行，没有WHEN语句则激活后立即执行
- 触发动作体：可以是一个匿名PL/SQL过程块呼号对已创建存储过程的调用
  - 如果是行级，则可以使用NEW或OLD引用事件后的新值与前的旧值
  - 如果是语句级，就不能引用
  - 触发动作体执行失败，激活触发器的时间终止执行，目标表和其他影响对象不发生变化
- 多个触发器：BEFORE触发器、激活触发器的语句、AFTER触发器。同级县创建相互粗发

- ```
1 DROP TRIGGER <触发器名> ON <表名>;
```

关系数据理论

- 什么是范式：是符合某一种级别的关系模式的集合
- 低一级范式通过模式分解可以转换为若刚高一级的范式关系模式的结合，这个过程就叫做规范化

1. 关系模式与1NF

- 之前我们了解到，关系模式是一个五元组： $R(U, D, DOM, F)$ ，我们现在讨论模式设计，可以把关系模式看做一个三元组 $R < U, F >$ 。U是一组属性，F是属性组U上的一组数据依赖
- 1NF：第一范式，每个分量必须是不可分开的数据项
- 刚刚提到了数据依赖，什么是数据依赖？
 - 是一个关系内属性间的一种约束关系（是通过属性间值的相等与否体现出来的数据间联系）；是现实世界属性间相互联系的抽象，是数据的内在性质和语义的表现
 - 主要的数据依赖：函数依赖FD和多值依赖MVD
 - 函数依赖的例子：
 - 上述关系模式存在问题：
 1. 数据冗余，这个不用说，很多属性间有一一对应的关系
 2. 修改复杂，更新异常，这是由数据冗余带来的，修改一个系主任的名字，就要改该系所有学生的元组
 3. 插入异常，删除异常：这两个问题是由于属性间的不必要的依赖关系带来的
 - 解决方法：拆成三个关系模式

前	后
$F=\{Sno \rightarrow Sdept, Sdept \rightarrow Mname, (Sno, Cno) \rightarrow Grade\}$	$S(Sno, Sdept, Sno \rightarrow Sdept);$
$SC(Sno, Cno, Grade, (Sno, Cno) \rightarrow Grade);$	
$DEPT(Sdept, Mname, Sdept \rightarrow Mname);$	

2. 具体讲一下函数依赖

- 定义就类似于数学中函数的定义，函数依赖是一个语义范畴
- 平凡函数依赖和非平凡函数依赖：平凡函数依赖是显然成立的事情（因为相当于冗余条件推已知条件），所以一般只讨论非平凡函数
- 完全函数依赖和部分函数依赖：就是看左边有没有冗余条件，有就是部分函数依赖，没有就是完全函数依赖
- 传递函数依赖：好理解，就依赖两次

3. 码

- 如果 $R < U, F >$ ，中U完全函数依赖与一个属性或属性组K，则K是R的候选码；如果部分函数依赖，则K是R的超码（候选码的超集是超码），有多个候选码，选一个当主码
- 主属性、非主属性：包含在任何一个候选码里的属性是主属性，反之是非主属性
- 全码：整个属性组是码
- 外码：不是当前模式的吗，但是是另外一个模式的码

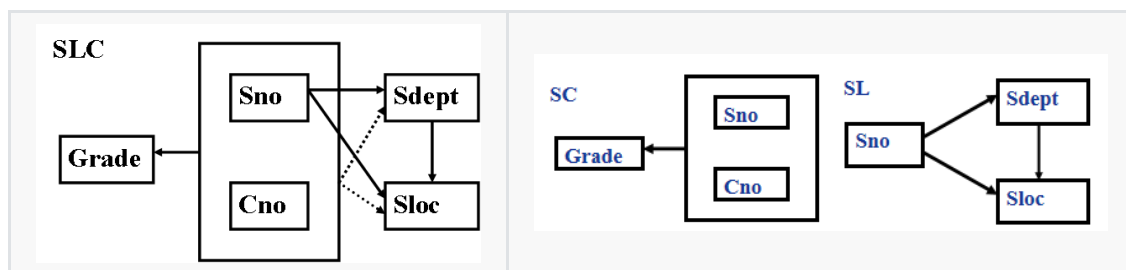
4. 正式进入范式的理解

4.1 1NF

- 所有属性都是不可分的
- 第一范式是对关系模式的最起码要求

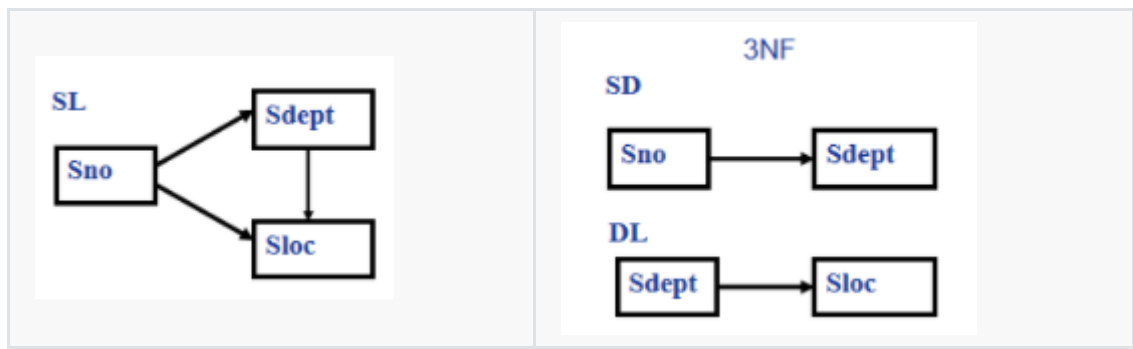
4.2 2NF

- 在1NF的基础上，每一个非主属性都完全函数依赖于任何一个候选码
- 判断方法，先找码，在判断主属性和非主属性，再判断依赖关系
- 问题的存在同之前提到的四个，解决方法是用投影分解把关系模式分解



4.3 3NF

- 每一个非主属性既不传递依赖于码，也不部分依赖于码



4.4 BCNF

- 每一个决定属性集都包含候选码
 - 所有非主属性都完全函数依赖于每个候选码 (2NF)
 - 所有非主属性都完全函数依赖于每个不包含它的候选码 (这句和放屁有什么区别)
 - 没有任何属性完全函数依赖于非码的任何一组属性 (?)
- 如果所有关系模式都属于BCNF, 那在函数依赖范畴里它已经实现了模式的彻底分解, 达到了最高的规范化长度, 消除了插入异常和删除异常
 - 3NF的不彻底性在于主属性可能对码的部分依赖和传递依赖

5. 打住打住, 先来看一下多值依赖, 在继续范式的推进

- 我们谈到, BCNF已经在函数范畴内实现了模式的彻底分解, 后面的工作其实是针对于多值依赖的解决
- 多值依赖: 将一个属性集分为三个部分XYZ, 对于任意关系, 给定一堆(x,z), 有一组Y的值对应, 仅仅取决于x值而与z值无关
 - 平凡多值依赖, Z为空集, 显然成立
 - 非平凡多值依赖
- 等价定义不放了, 感觉就是说属性值之间的关系, 类似于排列组合的味道 (ch36p6, p186)
- 多值依赖的性质:
 - 对称性、传递性、函数依赖是多值依赖的特殊情况 (我去! 哦好像也没啥)
 -
- 函数依赖和多值依赖的区别
 - 多值依赖的有效性与属性集的范围有关, 因为还涉及到其余属性Z
 - 函数依赖仅取决于XY两个属性集的值

好! 让我们继续范式的讨论, 下面是4NF

- 在1NF的基础上, 如果每个非平凡多值依赖, X都有码, 则为4NF
 - 其限制关系模式属性之间不能有非平凡的且非函数依赖的多值依赖, 也就是说4NF允许的非平凡多值依赖实际上就是函数依赖

6. 快奔溃了？别着急，为你准备了规范化小结！快来看看吧

- 规范化的基本思想就是逐步消除数据依赖中不合适的部分，达到各**关系模式**的**分离**。规范化在实质上是**概念的单一化**
-

草 这里大概丢了7000字的内容，typora为什么会崩啊，自动保存只能回溯到这个版本了

草草草

先继续吧x，刚刚看到死锁的预防

- 死锁的预防：破坏产生死锁的条件
 - 一次封锁法：每个事务必须一次将所有要使用的数据全部加锁，否则不能继续执行
 - 降低了系统并发度
 - 事先精确确定封锁对象是困难的
 - 顺序封锁法：预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁
 - 维护成本高
 - 难以实现
- 死锁的诊断：在解决死锁问题上更普遍采用的是诊断并解除死锁的方法
 - 超时法：如果一个事务的等待时间超过了规定时限，就认为发生了死锁
 - 简单，但可能误判
 - 等待图法：并发控制子系统周期性生成事务等待图，如果图中产生回路，就表示出现了死锁
- 解除死锁
 - 选择一个处理死锁代价最小的事务，将其撤销
 - 释放此事务持有的所有锁，继续运行其他事务

事务调度

- 对并发事务的不同调度会产生不同的结果。执行结果等价于串行调度的调度是正确的，称为可串行化调度。
- 可串行化性是并发事务正确调度的准则
- 冲突可串行化：是一个比可串行化更严格的条件
 - 冲突操作是指不同的事务对同一个数据的读写操作和写写操作，其他操作时不冲突操作
 - 同一事务的两个操作和不同事物冲突操作是不能交换的操作
- 一个调度 Sc 在保证冲突操作的次序不变的情况下，通过交换两个事务不冲突操作的次序得到另一个调度 Sc' ，如果 Sc' 是串行的，称调度 Sc 是冲突可串行化的调度

$Sc_1 = r_1(A)w_1(A)r_2(A)w_2(A)r_1(B)w_1(B)r_2(B)w_2(B)$

$Sc_2 = r_1(A)w_1(A)r_1(B)w_1(B)r_2(A)w_2(A)r_2(B)w_2(B)$

T_1 T_2

- Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 冲突可串行化的调度

- 冲突可串行化调度是可串行化调度的重逢条件，不是必要条件

两段锁协议

- 普遍采用两段锁方法实现并发调度的可串行性，是指所有事务必须分两个阶段对数据项加锁和解锁
 - 对任何数据进行读、写操作之前，事务首先要获得对该数据的封锁
 - 释放一个封锁后，事务不再申请和获得其他封锁
 - emmm就是说 对锁的申请和对锁的释放是分开来的，不能申请一个释放一个在申请一个
- 第一阶段：获得封锁，扩展阶段
- 第二阶段：释放封锁，收缩阶段
- 两段锁协议的事务可能发生死锁

封锁粒度

- 封锁对象的大小称为封锁粒度
- 封锁的对象是逻辑单元与物理单元
- 封锁粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也最小
- 多粒度封锁：支持多种封锁粒度

- 多粒度数：以树结构来表示多级封锁粒度，根节点是整个数据库，是最大的封锁粒度
- 多粒度封锁协议：
 1. 允许多粒度树中的每个节点被独立地加锁
 2. 对一个节点的加锁意味着整个节点的所有和后裔节点也加以同样的锁
 3. 在多粒度封锁汇总 一个数据对象可能以两种方式的浓缩（显式与隐式，上一条带来的结果），效果是一样的
- 隐式封锁带来，检查冲突时要看显式和隐式，对对象加锁时**要上中下都看**

意向锁

- 目的：提高某个数据对象加锁时系统的检查效率
- 如果对一节点加意向锁，则说明该节点的下层节点正在被加锁；对任一节点加及本所，都必须先对上层节点加意向锁
- 常用意向锁
 1. 意向共享锁：IS锁，对一个数据对象加IS锁，表示它的后裔节点一项加S锁
 2. 意向排它锁：IX锁，.....，意向加X锁
 3. 共享意向排他锁：SIX锁，对它加S锁，再加IX锁

4.

$T_1 \backslash T_2$	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

Y=Yes，表示相容的请求

N=No，表示不相容的请求

(a) 数据锁的相容矩阵